

LE MANUEL DU GATING

Politique de confiance, validation humaine et *human-in-the-loop* avec LangGraph

EPIC 4 — ACTIONPROPOSAL · POLICY · INTERRUPT · FILE DE VALIDATION

Édition de référence technique — architecture et code anticipés avant implémentation. Conçu pour impression noir & blanc haute résolution : hiérarchie par le gras, encadrés à filet, diagrammes vectoriels.

Document	Manuel du Gating — Edition 1.0
Périmètre fonctionnel	Epic 4 du backlog : module <code>gating/</code> , extension de <code>agent/</code>
Statut du code source	Non implémenté à ce jour — <code>gating/__init__.py</code> et <code>audit/__init__.py</code> seuls présents. Ce manuel documente l'architecture anticipée , cohérente avec le code déjà en place (Epic 1-3) et avec le backlog (US-401 à US-406).
Public visé	Développeur full-stack senior, nouveau sur RAG / agents / LangGraph — ce manuel enseigne les concepts au fil du texte.
Pré-requis	Lecture recommandée au préalable : <code>orchestrator-field-manual.md</code> (Epic 3) — ce manuel en est la suite directe et réutilise son vocabulaire (State, Node, Checkpointer...).
Stack couverte	Python 3.14, Pydantic v2, SQLAlchemy 2.0 async, Alembic, FastAPI, LangGraph (<code>interrupt</code> , <code>Command</code>), PostgreSQL
Diagrammes	7 — architecture, machine à états, 2 diagrammes de classes, 2 diagrammes de séquence, 1 diagramme d'activité

Format d'impression

Monochrome strict — aucune information portée par la couleur. A4, marges 18/15 mm.

MINDOPS · AGENT IA MODULAIRE (POC RÉUTILISABLE) · USAGE INTERNE

TABLE DES MATIÈRES

01	Vue d'ensemble	Pourquoi le gating existe, où il se situe dans l'architecture hexagonale, diagramme des modules
02	Concepts nouveaux	Human-in-the-loop, <code>interrupt()</code> , <code>Command(resume=...)</code> — la pièce manquante d'Epic 3
03	Le modèle ActionProposal	<code>gating/models.py</code> — table, statuts, diagramme de machine à états
04	La politique de confiance	<code>gating/policy.py</code> — <code>Decision</code> , <code>evaluate()</code> , configuration par type d'action
05	L'outil sensible	<code>agent/tools/send_email.py</code> — un <code>Tool</code> qui ne fait jamais l'action lui-même
06	Le graphe augmenté	<code>orchestrator.py</code> — le nœud <code>call_tools</code> redessiné, deux diagrammes de séquence
07	La file & l'API	<code>queue_service.py</code> , <code>router.py</code> , l'événement SSE <code>pending_approval</code>
08	Diagrammes de classes	Domaine (ActionProposal/Decision/policy) et intégration (Tool/services)
09	Diagramme d'activité	Le parcours complet d'une action, en couloirs, des trois niveaux d'autonomie
10	Stratégie de test	Tester une interruption sans file d'attente réelle ni LLM réel
11	Configuration & migration	<code>.env</code> , <code>config.py</code> , migration Alembic
12	Ce qui vient ensuite	Epic 5 (audit complet), Epic 6 (tableau de démonstration)
§	Glossaire	Tous les termes nouveaux introduits, A → Z

Comment lire ce manuel. Chaque module suit la même charpente : un rappel de *pourquoi*, le code proposé, un ou plusieurs diagrammes, puis un encadré **Vocabulaire**. Les encadrés à double filet marqués ★ **NOUVEAU CONCEPT** signalent une notion inédite pour quelqu'un venant du développement web classique — c'est le fil pédagogique du document. Les extraits de code respectent scrupuleusement les conventions déjà en place dans `dev/app/` (Pydantic v2, SQLAlchemy 2.0 `Mapped`, injection de dépendances FastAPI) : ce ne sont pas des exemples jetables, mais une proposition d'implémentation directement transposable.

Vue d'ensemble

Pourquoi le gating existe, où il se situe dans l'architecture hexagonale, ce qu'il ajoute concrètement au système existant.

1.1 — Le problème que résout le gating

Depuis l'Epic 3, l'agent MindOps sait faire une seule chose activement : **lire**. Son unique outil, `search_knowledge`, interroge la base de connaissances et ne modifie jamais rien — il peut donc s'exécuter immédiatement, sans risque, à chaque fois que le LLM le demande. Ce confort disparaît dès qu'un outil **agit dans le monde** : envoyer un email, écrire un ticket, déclencher un paiement. Un LLM peut halluciner un destinataire, mal interpréter une consigne, ou simplement se tromper — et une action envoyée ne se rappelle pas.

Le module `gating` (Epic 4 du backlog, US-401 à US-406) répond à un besoin précis : permettre à **chaque type d'action** de choisir, via une simple configuration, son niveau d'autonomie — jamais réécrire le code pour changer de politique. C'est la promesse centrale du projet, déjà énoncée dans la vision du backlog : « *le niveau d'autonomie devient un paramètre de configuration, pas une réécriture* ».

1.2 — Trois niveaux d'autonomie, une seule fonction

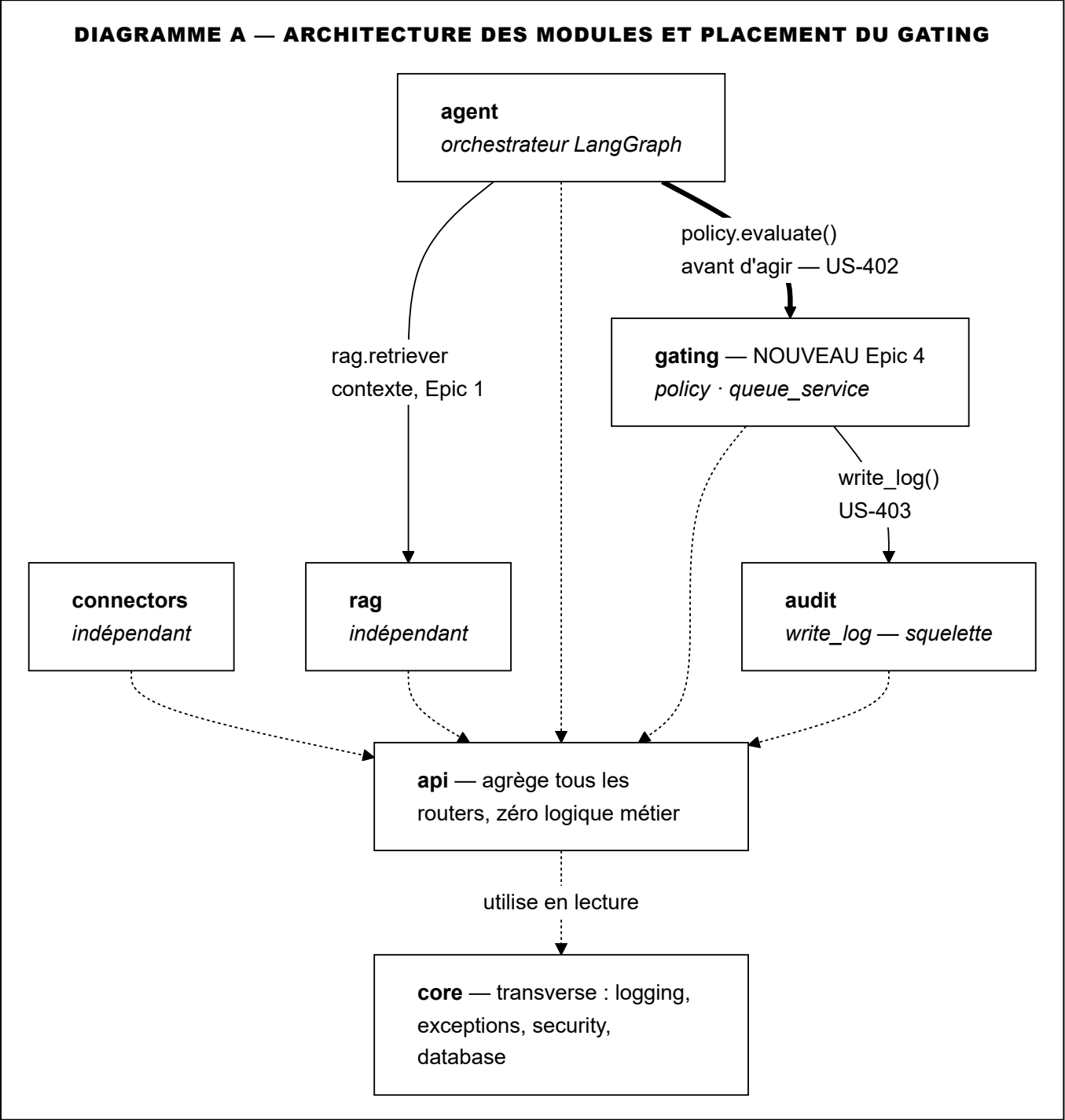
Le cœur du module tient en une fonction : `gating.policy.evaluate(action_type, confidence) -> Decision`. Elle renvoie toujours l'une de ces trois réponses :

Décision	Comportement	Coût / risque
<code>suggest_only</code>	L'agent formule la proposition en texte, mais n'exécute rien et ne crée même pas de ligne en base — c'est une réponse conversationnelle ordinaire.	Nul. Aucune trace persistée hors la conversation elle-même.
<code>require_validation</code>	Une <code>ActionProposal</code> est créée au statut <code>pending</code> , le graphe LangGraph s'interrompt , et l'exécution réelle attend une décision humaine via <code>POST /gating/{id}/decide</code> .	Faible. L'action ne part jamais sans un humain dans la boucle.
<code>auto_execute</code>	Une <code>ActionProposal</code> est créée déjà au statut <code>approved</code> , exécutée immédiatement, puis marquée <code>executed</code> .	Assumé. Réservé aux types d'action et aux niveaux de confiance jugés sûrs.

Ce module 01 pose le décor ; les modules 03 et 04 détaillent le modèle de données et la fonction `evaluate()` elle-même.

1.3 — Placement dans l'architecture hexagonale

Le projet suit une règle de dépendance stricte, déjà en vigueur depuis l'Epic 0 : `connectors` et `rag` sont indépendants ; `gating` dépend seulement d'`audit` ; `agent` dépend de `rag` et de `gating` ; `api` agrège tout sans logique métier ; `core` est transverse. Le diagramme ci-dessous situe le nouveau module et ses deux seules connexions sortantes.



Trait plein fin = appel direct. Trait épais = lien nouveau introduit par l'Epic 4. Pointillé = agrégation (api) ou usage transverse (core). Le module `gating` ne dépend que d'`audit` — il ne connaît ni `rag`, ni `connectors`, ce qui le rend testable et remplaçable seul.

NOTE — CE QUE CE MODULE NE FAIT PAS

Le gating ne décide jamais *quelle* action l'agent doit proposer — c'est le rôle du LLM et du prompt système. Il décide uniquement, une fois qu'une action est proposée, *si* et *quand* elle peut s'exécuter. C'est un pare-feu, pas un cerveau.

VOCABULAIRE DU MODULE 01

ActionProposal — l'enregistrement d'une action que l'agent souhaite exécuter, avec son statut de validation. Détaillé au module 03.

Politique de confiance — la règle, configurable par type d'action, qui détermine le niveau d'autonomie accordé.

Outil sensible — un `Tool` dont l'exécution produit un effet de bord réel dans le monde extérieur (envoyer, écrire, payer) par opposition à un outil en lecture seule.

Concepts nouveaux — Human-in-the-loop avec LangGraph

La pièce que l'Epic 3 avait anticipée sans l'utiliser : interrompre un graphe en plein milieu de son exécution, et le reprendre plus tard exactement là où il s'est arrêté.

2.1 — Pourquoi une boucle `for` classique ne suffit plus

Une boucle agentique écrite à la main (appeler le LLM, exécuter l'outil demandé, reboucler) fonctionne très bien tant que rien n'a besoin d'attendre un humain. Dès qu'une étape doit suspendre son exécution — parfois pendant plusieurs heures, le temps qu'une personne consulte sa file de validation — il faut soit bloquer un processus serveur (inacceptable, un serveur web ne peut pas geler une requête HTTP pendant des heures), soit réécrire à la main un mécanisme de file d'attente avec sondage (*polling*) : sauvegarder l'état de la conversation quelque part, arrêter la requête, puis, plus tard, retrouver cet état et reprendre le calcul exactement où il s'était arrêté. C'est un problème de **persistance d'état** à part entière, indépendant du problème métier.

★ NOUVEAU CONCEPT — INTERRUPT() — LA PAUSE NATIVE DE LANGGRAPH

LangGraph expose une fonction `interrupt(value)` (module `langgraph.types`), callable **depuis l'intérieur d'un nœud**. Son comportement est inhabituel pour qui n'a jamais manipulé de machine à états persistée : à la première exécution du nœud, elle lève une exception spéciale qui remonte jusqu'au `checkpoint` — celui-là même qui, depuis l'Epic 3, sauvegarde déjà l'historique de conversation (voir `agent/memory.py`). Le graphe s'arrête net, à l'intérieur du nœud, et rend la main à l'appelant avec la valeur passée à `interrupt()`. Rien n'est perdu : tout l'état du graphe — y compris les variables locales du nœud jusqu'à ce point — est figé dans le checkpoint.

2.2 — Reprendre l'exécution avec `Command(resume=...)`

Pour reprendre un graphe interrompu, on ne rappelle pas `invoke()` avec de nouveaux messages comme pour un tour de conversation normal. On appelle le graphe avec un objet `Command(resume=<valeur>)` et **le même** `config` (même `thread_id`) qu'à l'appel initial. LangGraph retrouve le checkpoint, ré-exécute le nœud interrompu depuis son tout début — mais cette fois, l'appel à `interrupt()` ne lève plus rien : il **renvoie directement** la valeur passée à `resume`, comme un appel de fonction normal qui aurait toujours retourné cette valeur. Le code du nœud continue alors son exécution après ce point.

```
# Extrait pédagogique – pas le code final, juste le mécanisme brut
from langgraph.types import interrupt, Command

def call_tools(state):
    ...
```

```

if tool.sensitive:
    decision = interrupt({"proposal_id": proposal.id, "action_type": tool.name})
    # --- première exécution : le code ne va jamais plus loin que la ligne ci-dessus ---
    # --- lors de la reprise : `decision` contient la valeur passée à Command(resume=...) ---
    if not decision["approved"]:
        return {"messages": [ToolMessage(content="Action rejetée.", ...)]}
result = await tool.execute(**call["args"])
...

# Première invocation : s'arrête à interrupt(), ne retourne jamais de réponse finale
await app.ainvoke(inputs, config=config)

# Reprise, arbitrairement plus tard, potentiellement depuis une autre requête HTTP :
await app.ainvoke(Command(resume={"approved": True}), config=config)

```

2.3 — Deux façons de déclencher une interruption

Mécanisme	Fonctionnement	Granularité
<code>interrupt_before=</code> <code>["tools"]</code> <i>option statique, passée à <code>graph.compile()</code></i>	Le graphe s'arrête toujours juste avant d'entrer dans le nœud nommé, quel que soit l'outil appelé.	Par nœud. Interromptrait aussi <code>search_knowledge</code> , ce qu'on ne veut pas.
<code>interrupt()</code> dynamique <i>appel conditionnel, à l'intérieur du code du nœud</i>	Le nœud décide lui-même, appel par appel, s'il doit s'arrêter — ici, seulement si <code>tool.sensitive</code> est vrai.	Par outil, voire par valeur de paramètre. C'est le choix retenu pour ce manuel.

Le module 06 montre pourquoi la version dynamique est la seule cohérente avec une politique « *par type d'action* » (US-402) : `search_knowledge` doit continuer à s'exécuter sans jamais interrompre le graphe, alors que `send_email` doit pouvoir s'arrêter — la décision se prend donc outil par outil, à l'intérieur d'un seul et même nœud `call_tools`.

⚠

ATTENTION — IDEMPOTENCE OBLIGATOIRE

Comme le nœud est **rejoué depuis son début** à la reprise, tout code placé *avant* l'appel à `interrupt()` s'exécute une seconde fois. Créer l'`ActionProposal` et évaluer la politique avant l'appel à `interrupt()` est donc dangereux si ce n'est pas protégé — d'où la nécessité, au module 05, de vérifier qu'une proposition n'existe pas déjà pour cet appel d'outil avant d'en recréer une.

VOCABULAIRE DU MODULE 02

human-in-the-loop — un système où une décision humaine explicite est requise avant qu'une étape automatisée ne se poursuive.

`interrupt()` — fonction LangGraph qui suspend le nœud courant, persiste son état via le checkpointing, et renvoie la main à l'appelant.

`Command(resume=...)` — objet utilisé pour reprendre un graphe interrompu, en réinjectant une valeur à l'endroit exact de la pause.

rejeu (replay) — le fait, pour `LangGraph`, de ré-exécuter le code d'un nœud depuis son début lors d'une reprise — jusqu'à atteindre l'appel à `interrupt()` qui, cette fois, ne bloque plus.

Le modèle ActionProposal

`gating/models.py` — US-401. La trace d'une action, de sa naissance à son exécution ou son rejet.

3.1 — Le modèle SQLAlchemy

Même style que `rag/models.py` : `Mapped[...]`, `mapped_column`, héritage de `Base` défini dans `core/database.py`. Le statut est un `StrEnum` Python, stocké comme chaîne (`native_enum=False`) plutôt que comme type `ENUM` PostgreSQL — cohérent avec un projet qui doit rester simple à faire évoluer sans migration lourde à chaque nouveau statut.

`gating/models.py`

```
from datetime import datetime
from enum import StrEnum

from sqlalchemy import JSON, Enum as SAEnum, func
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base

class ActionStatus(StrEnum):
    PENDING = "pending"
    APPROVED = "approved"
    REJECTED = "rejected"
    EXECUTED = "executed"

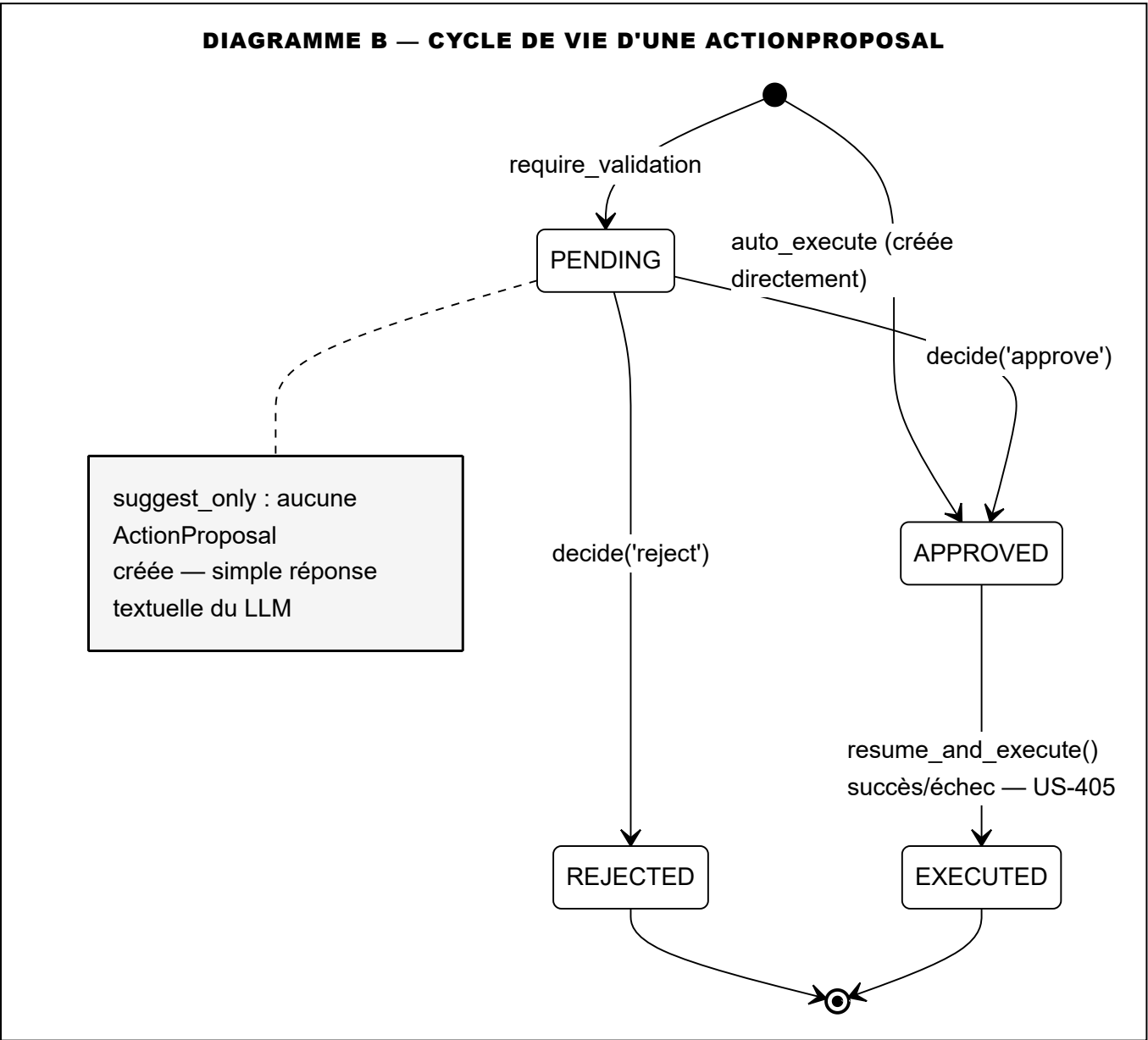
class ActionProposal(Base):
    __tablename__ = "action_proposals"

    id: Mapped[int] = mapped_column(primary_key=True)
    action_type: Mapped[str] = mapped_column() # ex: "send_email" – nom du Tool concerné
    parameters: Mapped[dict] = mapped_column(JSON) # args validés par le args_schema de l'outil
    status: Mapped[ActionStatus] = mapped_column(
        SAEnum(ActionStatus, native_enum=False), default=ActionStatus.PENDING
    )
    conversation_id: Mapped[str] = mapped_column() # = thread_id LangGraph, pour reprendre le bon
    graph # graphe
    tool_call_id: Mapped[str] = mapped_column() # id de l'appel outil LLM, pour réémettre le bon
    ToolMessage
    result: Mapped[str | None] = mapped_column(default=None)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
    decided_at: Mapped[datetime | None] = mapped_column(default=None)
    executed_at: Mapped[datetime | None] = mapped_column(default=None)
```

Deux champs méritent une explication : `conversation_id` et `tool_call_id` ne figurent pas dans les critères d'acceptation littéraux du backlog, mais sont indispensables au mécanisme de reprise du module 06 — sans `conversation_id`, `POST /gating/{id}/decide` ne saurait pas *quel* graphe

LangGraph reprendre (quel `thread_id` passer au `config`) ; sans `tool_call_id`, il ne saurait pas à quel appel outil du LLM associer le `ToolMessage` de résultat.

3.2 — Machine à états



Deux portes d'entrée (`auto_execute` crée directement au statut `APPROVED`, sans jamais passer par `PENDING`), un seul état terminal en cas de refus, et `suggest_only` qui ne produit aucune ligne en base — le chemin le plus économe.

VOCABULAIRE DU MODULE 03

StrEnum — énumération Python (3.11+) dont chaque membre se comporte aussi comme une chaîne de caractères, pratique pour la sérialisation JSON et le stockage en base sans conversion manuelle.

colonne JSON — type de colonne SQLAlchemy stockant un objet Python arbitraire (dict, liste) sérialisé, ici utilisé pour les `parameters` d'une action dont la forme diffère selon l'outil.

machine à états — une modélisation où une entité ne peut se trouver que dans un nombre fini de statuts, avec des transitions explicites entre eux — ici `pending → approved → executed` ou `pending → rejected`.

La politique de confiance

`gating/policy.py` — US-402 et US-406. Une fonction pure, une configuration, zéro redéploiement pour changer de comportement.

4.1 — `evaluate()`, une fonction délibérément pure

Aucun accès base de données, aucun appel réseau : `evaluate()` lit uniquement la configuration chargée en mémoire (`get_settings()`, déjà en place depuis l'Epic 0 via `pydantic-settings`). C'est ce qui la rend triviale à tester avec les trois décisions possibles, comme l'exige explicitement le critère d'acceptation de l'US-402.

`gating/policy.py`

```
from enum import StrEnum

from app.config import get_settings

class Decision(StrEnum):
    SUGGEST_ONLY = "suggest_only"
    REQUIRE_VALIDATION = "require_validation"
    AUTO_EXECUTE = "auto_execute"

def evaluate(action_type: str, confidence: float) -> Decision:
    """Politique de confiance : configurable par type d'action, sans redéploiement (US-406)."""
    settings = get_settings()
    configured = settings.gating_policy.get(action_type, Decision.REQUIRE_VALIDATION)
    decision = Decision(configured)

    # Garde-fou : même autorisé en auto_execute, un outil sous le seuil de confiance
    # configuré retombe en validation humaine plutôt que de s'exécuter à l'aveugle.
    if decision is Decision.AUTO_EXECUTE and confidence < settings.gating_min_confidence:
        return Decision.REQUIRE_VALIDATION
    return decision
```

4.2 — La configuration qui pilote tout

Deux nouveaux réglages dans `config.py`, suivant exactement le style déjà utilisé pour `rag_chunk_max_tokens` ou `embedding_provider` : un `Field` Pydantic avec son `alias` de variable d'environnement.

```
gating_policy: dict[str, str] = Field(
    default_factory=lambda: {"send_email": "require_validation"}, alias="GATING_POLICY"
)
gating_min_confidence: float = Field(default=0.8, alias="GATING_MIN_CONFIDENCE")
```

★ **NOUVEAU CONCEPT — PYDANTIC-SETTINGS ET LES TYPES COMPLEXES EN VARIABLE D'ENVIRONNEMENT**

`pydantic-settings` sait désérialiser automatiquement un `dict` ou une `list` depuis une variable d'environnement contenant du JSON brut. Concrètement, dans `.env` :

```
GATING_POLICY={"send_email": "require_validation", "create_ticket": "auto_execute"}
```

Faire passer `send_email` de `require_validation` à `auto_execute` — la démonstration clé exigée par l'US-406 — ne touche donc **aucune ligne de code Python**, uniquement cette variable d'environnement.

4.3 — Table de vérité

Type d'action	Politique configurée	Confiance de l'appel	Décision renvoyée
<code>send_email</code>	<code>require_validation</code>	—	<code>REQUIRE_VALIDATION</code>
<code>send_email</code>	<code>auto_execute</code>	0.95	<code>AUTO_EXECUTE</code>
<code>send_email</code>	<code>auto_execute</code>	0.4 (garde-fou)	<code>REQUIRE_VALIDATION</code>
type non configuré	— (valeur par défaut)	—	<code>REQUIRE_VALIDATION</code>

Remarque de conception : la valeur par défaut d'un type d'action inconnu est volontairement la plus prudente possible — fail safe, jamais fail open.

VOCABULAIRE DU MODULE 04

fonction pure — une fonction dont le résultat ne dépend que de ses arguments (et, ici, de la configuration statique), sans effet de bord ni état mutable.

fail safe — principe de conception où, en cas de doute ou de configuration manquante, le système choisit systématiquement le comportement le moins risqué.

garde-fou (guardrail) — une vérification supplémentaire qui limite un comportement autorisé, ici le seuil de confiance minimal appliqué même en `auto_execute`.

L'outil sensible

`agent/tools/send_email.py` — US-404. Un Tool qui ne fait jamais l'action lui-même en un seul appel.

5.1 — Ce qui ne change pas : le contrat `Tool`

Rappel du contrat posé dès l'Epic 3 (`agent/tools/base.py`) : une classe abstraite (`ABC`), trois attributs de classe (`name`, `description`, `args_schema`) et une méthode `execute()`. Le module 10 de l'Epic 3 promettait explicitement que ce contrat resterait stable — une seule addition : un attribut `sensitive`, à `False` par défaut, qui ne casse **aucun** outil existant.

`agent/tools/base.py` – diff minimal

```
class Tool(ABC):
    name: ClassVar[str]
    description: ClassVar[str]
    args_schema: ClassVar[type[BaseModel]]
    sensitive: ClassVar[bool] = False    # + NOUVEAU – outils en lecture seule : rien à changer

    @abstractmethod
    async def execute(self, **kwargs: object) -> str: ...
```

5.2 — `SendEmailTool`

La différence essentielle avec `SearchKnowledgeTool` : `execute()` ne contacte **jamais** directement Resend ou SendGrid. Il se contente de préparer les paramètres — l'envoi réel est délégué à une fonction séparée, appelée uniquement depuis le nœud `call_tools` de l'orchestrateur, après que la politique a tranché (module 06). C'est ce découplage qui rend le mécanisme d'idempotence du module 02 gérable : `execute()` lui-même reste un aller simple, sans branche interne dépendant d'un `interrupt()`.

`agent/tools/send_email.py`

```
from pydantic import BaseModel, Field

from app.agent.tools.base import Tool
from app.config import get_settings

class SendEmailArgs(BaseModel):
    to: str = Field(description="Adresse email du destinataire")
    subject: str = Field(description="Objet de l'email")
    body: str = Field(description="Corps du message, texte brut")

class SendEmailTool(Tool):
    """Outil sensible (US-404) : prépare l'envoi, ne l'exécute jamais lui-même."""
```

```

name = "send_email"
description = (
    "Envoie un email récapitulatif à un destinataire. Nécessite potentiellement "
    "une validation humaine selon la politique de confiance en vigueur."
)
args_schema = SendEmailArgs
sensitive = True

async def execute(self, *, to: str, subject: str, body: str) -> str:
    # Ce texte n'est jamais le résultat final : il est immédiatement remplacé,
    # au niveau du nœud call_tools, par le vrai résultat de l'envoi ou du rejet.
    return f"[proposition d'envoi préparée pour {to}]"

async def deliver(*, to: str, subject: str, body: str) -> str:
    """Envoi réel, isolé et remplaçable (US-404) – appelé uniquement après feu vert du gating."""
    settings = get_settings()
    if settings.email_api_key is None:
        return "échec : EMAIL_API_KEY non configurée"
    # Appel Resend/SendGrid réel ici – isolé pour rester substituable par un faux en test.
    return "envoyé"

```

🔪 NOTE — POURQUOI SÉPARER EXECUTE() ET DELIVER()

Le paramètre `settings.email_api_key` existe déjà dans `config.py` depuis l'Epic 0 — il attendait ce module. En isolant l'appel réseau réel dans `deliver()`, un test peut vérifier tout le cheminement de la politique de confiance (création de proposition, interruption, décision, reprise) sans jamais appeler un vrai fournisseur d'email, exactement comme `llm_client.py` est substitué par `ScriptedChatModel` dans les tests de l'Epic 3.

VOCABULAIRE DU MODULE 05

effet de bord (side effect) — toute modification visible en dehors du programme lui-même — envoyer un email, écrire en base, appeler une API tierce.

séparation préparation/exécution — un pattern où la décision de *quoi* faire (validée, sérialisable, rejouable) est séparée du *comment* le faire réellement (appel réseau, non rejouable sans risque de doublon).

Le graphe augmenté

`orchestrator.py` — là où toutes les pièces précédentes se rencontrent réellement.

6.1 — Le nœud `call_tools` redessiné

Une seule fonction change dans `orchestrator.py`. La structure du graphe (deux nœuds, `call_model` et `tools`, reliés par une arête conditionnelle) reste rigoureusement identique à l'Epic 3 — seul le **contenu** du nœud `tools` gagne une branche.

agent/orchestrator.py – `call_tools` étendu

```
from langgraph.types import interrupt

from app.gating import policy
from app.gating.models import ActionProposal, ActionStatus
from app.agent.tools.send_email import deliver

async def call_tools(state: MessagesState, *, db, conversation_id: str) -> dict:
    last_message = state["messages"][-1]
    results = []
    for call in last_message.tool_calls:
        tool = tools_by_name.get(call["name"])
        if tool is None:
            results.append(ToolMessage(content=f"Outil inconnu : {call['name']}",
            tool_call_id=call["id"]))
            continue

        if not tool.sensitive:
            content = await tool.execute(**call["args"]) # inchangé – chemin Epic 3
            results.append(ToolMessage(content=content, tool_call_id=call["id"]))
            continue

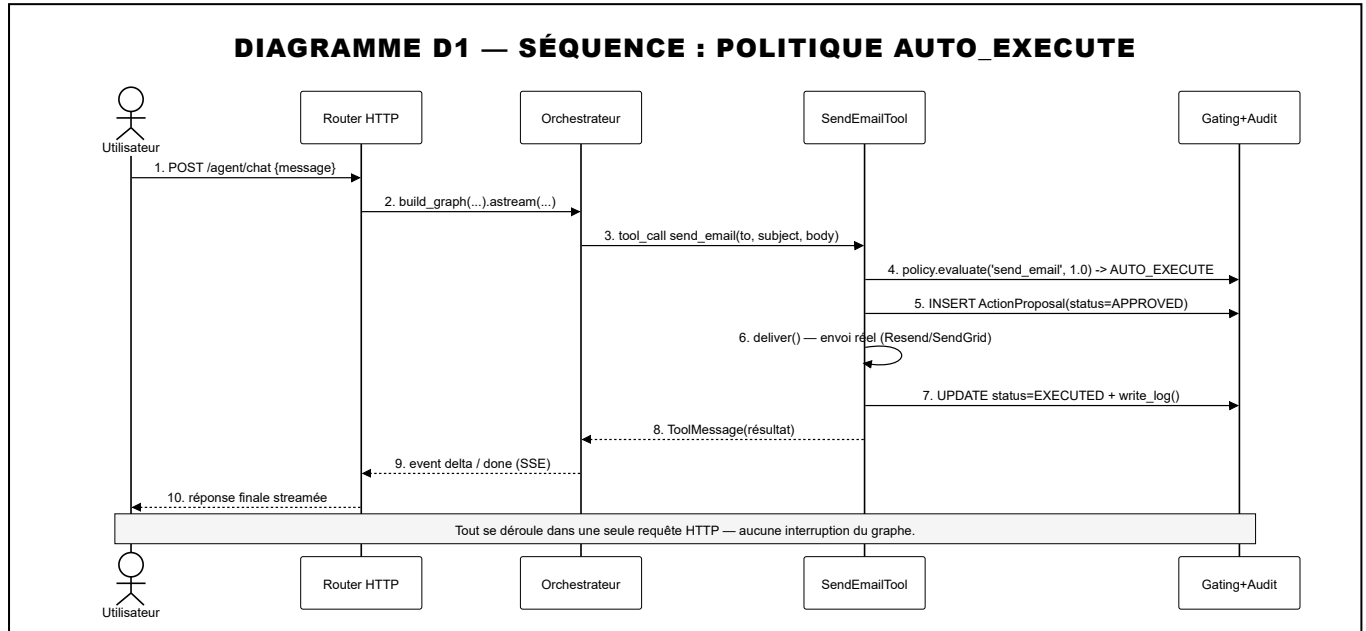
        # --- outil sensible : la politique tranche avant toute exécution réelle ---
        proposal = await _get_or_create_proposal(db, call, conversation_id, tool.name)
        decision = policy.evaluate(tool.name, confidence=1.0)

        if decision is policy.Decision.SUGGEST_ONLY:
            content = f"Suggestion : {tool.name}({call['args']}) – non exécutée (suggest_only)."
        elif decision is policy.Decision.REQUIRE_VALIDATION:
            outcome = interrupt({ # ★ pause ici – module 02
                "proposal_id": proposal.id, "action_type": tool.name, "parameters": call["args"],
            })
            content = await _apply_decision(db, proposal, outcome, call["args"])
        else: # AUTO_EXECUTE
            proposal.status = ActionStatus.APPROVED
            await db.commit()
            content = await _execute_and_log(db, proposal, call["args"])

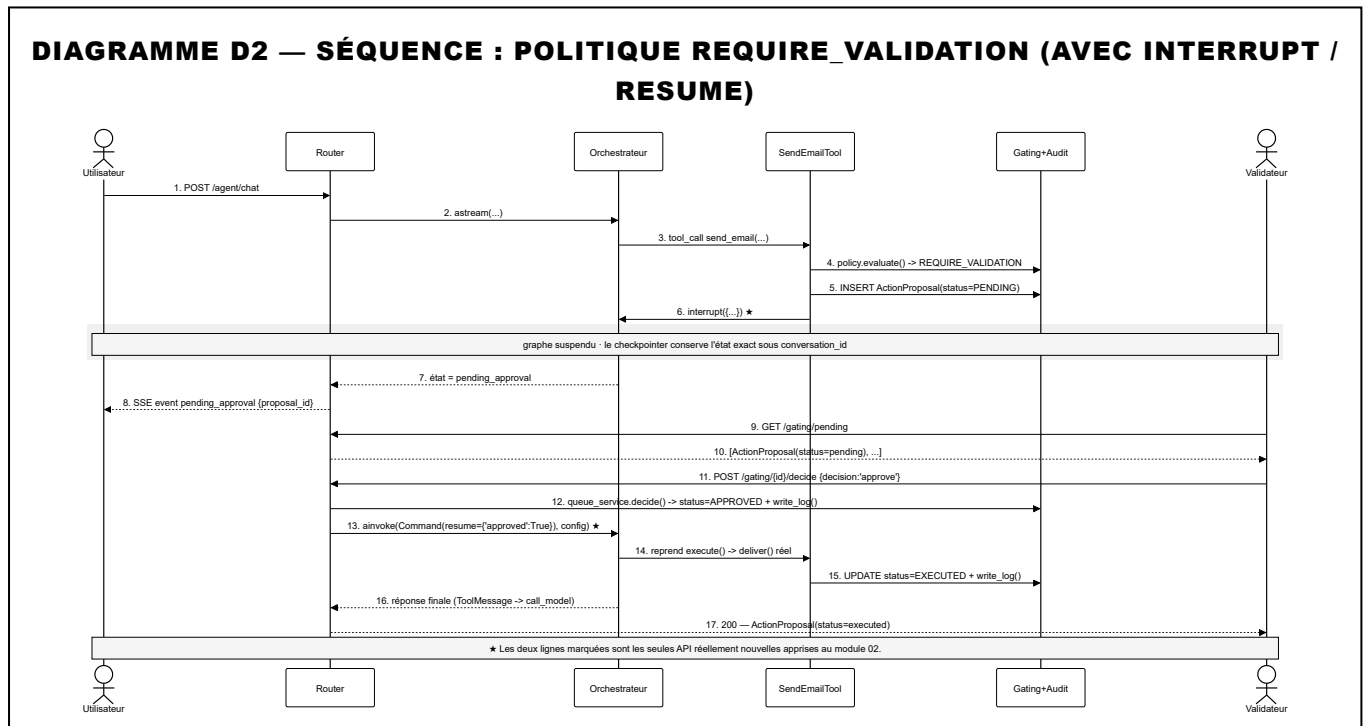
        results.append(ToolMessage(content=content, tool_call_id=call["id"]))
    return {"messages": results}
```

`_get_or_create_proposal` recherche d'abord une `ActionProposal` existante pour ce `tool_call_id` avant d'en créer une — c'est la protection contre le rejeu évoquée au module 02 : après une reprise, ce code s'exécute une seconde fois, mais ne doit jamais dupliquer la proposition initiale.

6.2 — Chemin 1 : `auto_execute`, sans jamais quitter la requête HTTP



6.3 — Chemin 2 : `require_validation`, en deux requêtes séparées



Entre les étapes 8 et 9, un délai arbitraire peut s'écouler — minutes ou heures. Le graphe ne consomme aucune ressource serveur pendant cette attente : il n'existe qu'en tant que ligne de checkpoint en base.

NOTE — LE CONTRAT EXTERNE NE CASSE PAS

`POST /agent/chat` garde exactement la même forme qu'en Epic 3. Il gagne seulement un événement SSE supplémentaire possible, `event: pending_approval` — exactement l'enrichissement anticipé dans le tableau du module 10 du manuel de l'orchestrateur.

VOCABULAIRE DU MODULE 06

lifeline (ligne de vie) — dans un diagramme de séquence, la ligne verticale pointillée représentant l'existence d'un participant au fil du temps.

appel synchrone / retour — flèche pleine pour un appel qui attend une réponse, flèche pointillée pour le message de retour correspondant.

requêtes découplées dans le temps — deux requêtes HTTP distinctes (l'appel initial, puis la décision) qui coopèrent sur un seul état logique persistant, sans connexion ouverte entre les deux.

La file de validation & l'API

`gating/queue_service.py`, `gating/router.py` — US-403. Ce que voit — et déclenche — la personne qui valide.

7.1 — `queue_service.py`

Toute la logique métier de la file vit ici, jamais dans le router (cohérent avec le reste du projet : comparer avec `rag/router.py`, qui ne fait qu'appeler `ingestion.py` et `retriever.py`).

`gating/queue_service.py`

```
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.audit.service import write_log
from app.gating.models import ActionProposal, ActionStatus

async def list_pending(db: AsyncSession) -> list[ActionProposal]:
    result = await db.execute(
        select(ActionProposal).where(ActionProposal.status == ActionStatus.PENDING)
    )
    return list(result.scalars())

async def decide(db: AsyncSession, proposal_id: int, decision: str) -> ActionProposal:
    proposal = await db.get(ActionProposal, proposal_id)
    if proposal is None or proposal.status != ActionStatus.PENDING:
        raise ValueError(f"Proposition {proposal_id} introuvable ou déjà tranchée.")

    proposal.status = ActionStatus.APPROVED if decision == "approve" else ActionStatus.REJECTED
    await db.commit()
    await db.refresh(proposal)
    await write_log("gating.decision", {"proposal_id": proposal_id, "decision": decision})
    return proposal    # US-405 : l'exécution effective est déclenchée par l'appelant (le router),
                        # via la reprise du graphe – voir module 06, étape 13
```

7.2 — `gating/router.py`

`gating/router.py`

```
from fastapi import APIRouter, Depends
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.database import get_db
from app.gating import queue_service
from app.gating.schemas import ActionProposalOut, DecisionIn

router = APIRouter()
```

```

@router.get("/pending", response_model=list[ActionProposalOut], summary="Lister les actions en
attente")
async def pending(db: AsyncSession = Depends(get_db)) -> list[ActionProposalOut]:
    return await queue_service.list_pending(db)

@router.post("/{proposal_id}/decide", response_model=ActionProposalOut, summary="Approuver ou
rejeter")
async def decide(
    proposal_id: int, payload: DecisionIn, db: AsyncSession = Depends(get_db)
) -> ActionProposalOut:
    proposal = await queue_service.decide(db, proposal_id, payload.decision)
    if payload.decision == "approve":
        await resume_agent_graph(proposal)    # module 06, étape 13 – reprend le graphe LangGraph
    return proposal

```

7.3 — Une dépendance minimale vers `audit`

La règle de dépendance du projet impose `gating → audit`. Or `audit/` n'existe encore que sous forme de `__init__.py` vide — son implémentation complète est l'Epic 5. Pour ne pas bloquer l'Epic 4 sur l'Epic 5, ce manuel propose un **squelette minimal**, suffisant pour satisfaire le critère d'acceptation de l'US-403 (« *toute décision est immédiatement enregistrée dans le module audit* »), et strictement compatible avec l'interface que l'Epic 5 viendra enrichir (US-501, US-502) sans casser aucun appelant.

```

# audit/service.py – squelette minimal Epic 4, complété en Epic 5 (AuditLog persistant, US-501)
from app.core.logging import logger

async def write_log(event_type: str, payload: dict) -> None:
    logger.info("audit_event", event_type=event_type, payload=payload)

```

🔪 NOTE — POURQUOI CE DÉCOUPAGE EST DÉLIBÉRÉ

C'est une démonstration concrète du principe Ports & Adapters : `queue_service.py` appelle `write_log()` sans jamais savoir si celle-ci écrit dans les logs structurés (aujourd'hui) ou dans une table `AuditLog` persistante (dès l'Epic 5). Le jour où l'Epic 5 remplace l'implémentation, **aucune ligne de `gating/` ne change**.

VOCABULAIRE DU MODULE 07

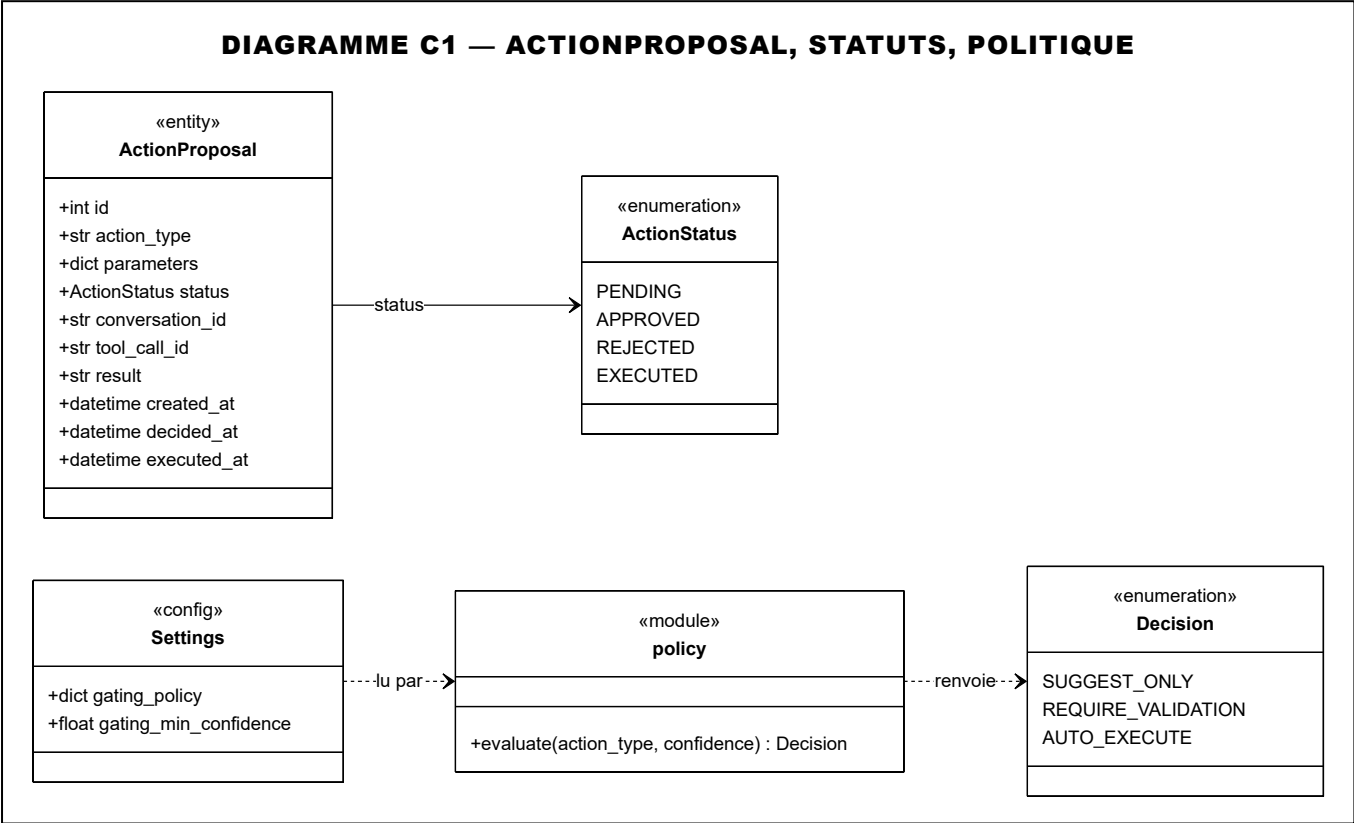
service (couche) — module regroupant la logique métier d'un domaine, appelé par le router mais indépendant du protocole HTTP.

squelette minimal (stub) — une implémentation volontairement incomplète mais fonctionnelle d'une interface, destinée à être remplacée plus tard sans changer son contrat.

Diagrammes de classes

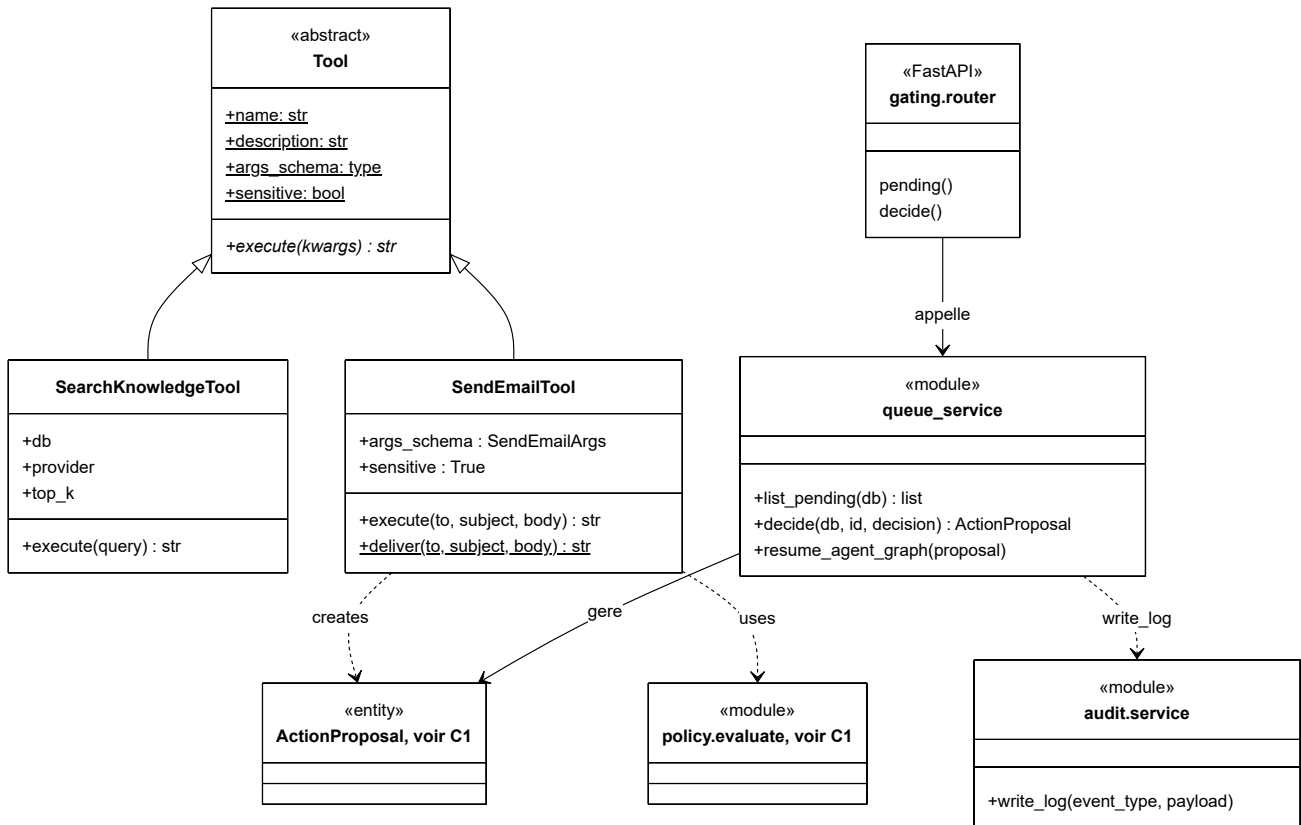
Deux vues complémentaires : le domaine du gating, puis son intégration avec l'existant.

8.1 — Domaine du gating



8.2 — Intégration outils & services

DIAGRAMME C2 — TOOL, SENDEMAILTOOL ET LES SERVICES DE GATING



Flèche pleine + triangle creux = héritage. Flèche pointillée = dépendance d'usage (« uses »/« creates »). Flèche pleine simple = association/gestion directe. Les cadres en tirets renvoient au diagramme C1 pour éviter la redondance.

VOCABULAIRE DU MODULE 08

stéréotype UML (« ... ») — une étiquette entre guillemets français indiquant la nature d'un élément —

«abstract», «enum», «module» — sans faire partie de son nom.

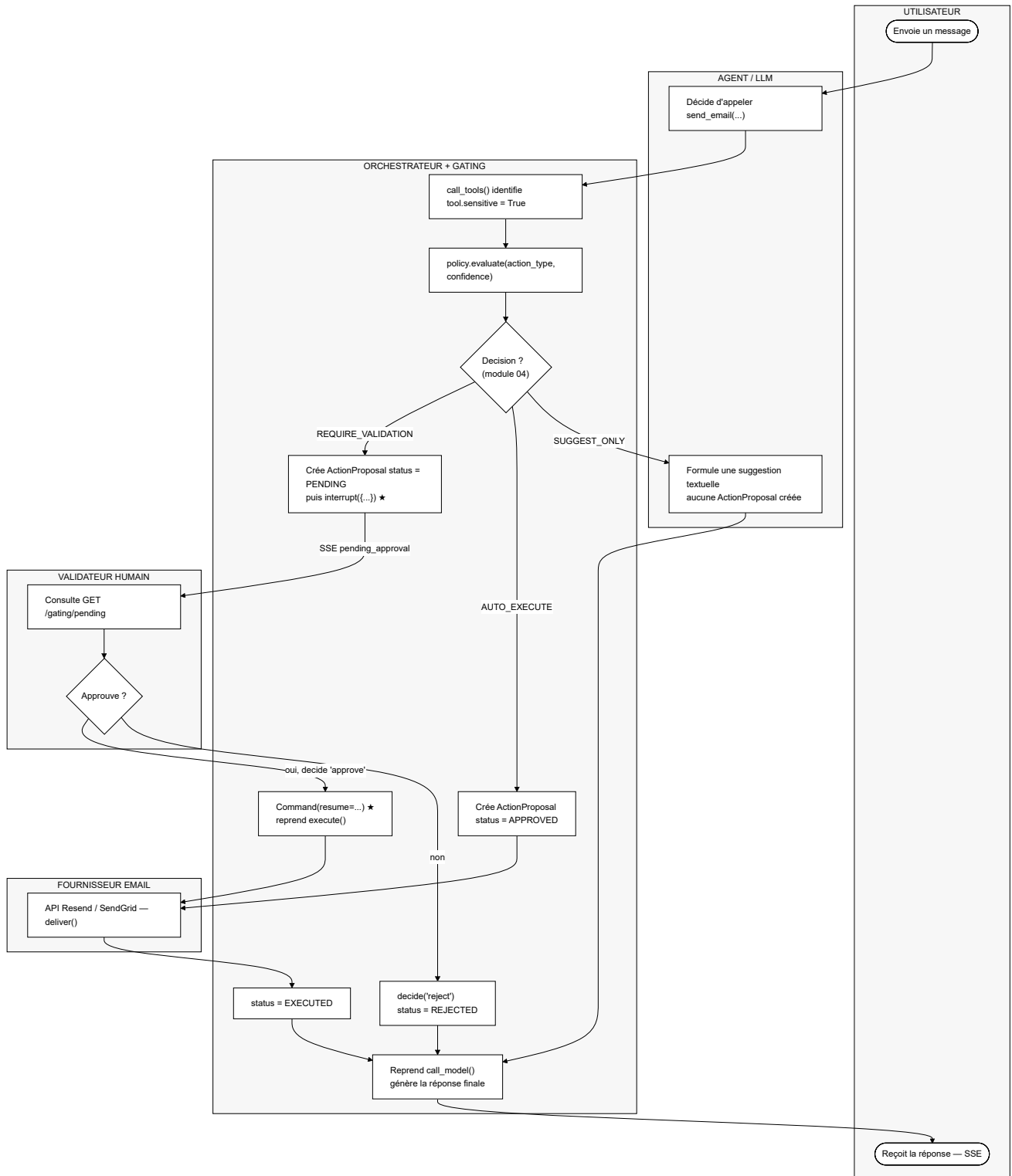
association vs dépendance — une association (trait plein) relie deux classes qui se connaissent durablement (l'une possède/gère l'autre) ; une dépendance (trait pointillé) indique un usage ponctuel, souvent le temps d'un seul appel de méthode.

Diagramme d'activité

Le parcours complet d'une action, en couloirs (swimlanes), des trois niveaux d'autonomie jusqu'à la réponse finale.

Ce diagramme se lit comme une carte : chaque colonne est un acteur responsable, chaque case ovale une activité, le losange une décision. Les trois branches issues du losange central correspondent exactement aux trois valeurs de `Decision` du module 04 — c'est la même règle de gestion, montrée cette fois comme un parcours plutôt que comme une table.

DIAGRAMME E — ACTIVITÉ COMPLÈTE, EN COULOIRS



VOCABULAIRE DU MODULE 09

couloir (swimlane) — une bande verticale ou horizontale regroupant les activités sous la responsabilité d'un même acteur.

nœud de décision — représenté par un losange, un point où le flux se divise selon une condition testée une seule fois.

convergence (merge) — point où plusieurs chemins alternatifs rejoignent un flux unique commun, ici avant la génération de la réponse finale.

Stratégie de test

Tester une interruption LangGraph sans file d'attente réelle, sans LLM réel, sans fournisseur d'email réel.

10.1 — Le même principe qu'en Epic 3, appliqué à l'interruption

Les tests de l'orchestrateur (Epic 3) reposent sur deux doublures : `ScriptedChatModel` (rejoue des réponses LLM prédéfinies) et `EchoTool` (un outil factice qui enregistre ses appels). Le module gating ajoute une seule doublure supplémentaire : un outil sensible factice, pour vérifier le cycle interruption/reprise sans dépendre de `SendEmailTool` ni d'un vrai fournisseur d'email.

tests/gating/fakes.py

```
from app.agent.tools.base import Tool
from pydantic import BaseModel

class FakeSensitiveArgs(BaseModel):
    value: str

class FakeSensitiveTool(Tool):
    """Outil sensible factice : mêmes garanties que SendEmailTool, sans réseau."""

    name = "fake_sensitive_action"
    description = "Outil de test, sensible, pour vérifier le cycle gating sans dépendance externe."
    args_schema = FakeSensitiveArgs
    sensitive = True

    def __init__(self):
        self.delivered: list[str] = []

    async def execute(self, *, value: str) -> str:
        self.delivered.append(value)
        return f"livré: {value}"
```

10.2 — Quatre cas de test qui couvrent tout le module

Scénario	Ce qu'il prouve
test_auto_execute_runs_without_interruption	Avec <code>GATING_POLICY=</code> <code>{"fake_sensitive_action": "auto_execute"}</code> , le graphe se termine en un seul <code>ainvoke()</code> , la proposition est créée directement <code>APPROVED</code> puis <code>EXECUTED</code> .
test_require_validation_pauses_the_graph	Avec la politique par défaut, <code>ainvoke()</code> retourne un état interrompu (<code>app.aget_state(config).next</code> non vide) — le graphe ne produit aucune réponse finale tant que la reprise n'a pas lieu.

<code>test_resume_after_approval_executes_the_tool</code>	Un second <code>ainvoke(Command(resume=...), config)</code> , avec le même <code>thread_id</code> , termine le graphe et déclenche réellement <code>execute()</code> — vérifié via <code>fake_tool.delivered</code> .
<code>test_rejection_never_calls_execute</code>	Une reprise avec <code>resume={"approved": False}</code> renvoie un <code>ToolMessage</code> de rejet, et <code>fake_tool.delivered</code> reste vide.

```
async def test_require_validation_pauses_the_graph(fake_tool: FakeSensitiveTool):
    llm = ScriptedChatModel(responses=[AIMessage(
        content="", tool_calls=[{"name": "fake_sensitive_action", "args": {"value": "x"}, "id":
"c1"}]
    )])
    app = build_graph(llm, [fake_tool], MemorySaver())
    config = {"configurable": {"thread_id": "gating-test-1"}}

    await app.ainvoke({"messages": [("user", "fais l'action sensible")]}, config=config)

    state = await app.aget_state(config)
    assert state.next != ()          # le graphe est bien suspendu, pas terminé
    assert fake_tool.delivered == [] # rien n'a été exécuté avant la validation humaine
```

⚠ ATTENTION — PIÈGE CLASSIQUE

Comme au module 02 : si le test appelle `ainvoke()` une seconde fois sans `Command(resume=...)` mais avec de nouveaux messages, LangGraph traiterai cela comme un **nouveau tour de conversation**, pas comme une reprise — le graphe resterait bloqué indéfiniment sur l'interruption initiale.

VOCABULAIRE DU MODULE 10

`state.next` — propriété exposée par `app.aget_state(config)`, listant les nœuds que LangGraph s'apprête à exécuter — vide seulement quand le graphe est réellement terminé.

doublure de test spécialisée — une fausse implémentation créée pour isoler un seul comportement à tester (ici, le cycle interruption/reprise), plutôt que de réutiliser une doublure généraliste comme `EchoTool`.

Configuration & migration

Les fichiers transverses à mettre à jour pour que tout ce qui précède compile et se déploie.

11.1 — `.env.example`, additions

```
GATING_POLICY={"send_email": "require_validation"}
GATING_MIN_CONFIDENCE=0.8

# Déjà présentes depuis l'Epic 0 (config.py), utilisées pour la première fois ici :
EMAIL_API_KEY=
EMAIL_FROM=
```

11.2 — Migration Alembic

Même structure que les migrations existantes (`alembic/versions/`), générée avec `alembic revision --autogenerate` une fois `gating/models.py` écrit.

```
"""add action_proposals table

Revision ID: <génééré>
Revises: <dernière révision rag>
"""
from alembic import op
import sqlalchemy as sa

def upgrade() -> None:
    op.create_table(
        "action_proposals",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("action_type", sa.String(), nullable=False),
        sa.Column("parameters", sa.JSON(), nullable=False),
        sa.Column("status", sa.String(), nullable=False, server_default="pending"),
        sa.Column("conversation_id", sa.String(), nullable=False),
        sa.Column("tool_call_id", sa.String(), nullable=False),
        sa.Column("result", sa.String(), nullable=True),
        sa.Column("created_at", sa.DateTime(), server_default=sa.func.now()),
        sa.Column("decided_at", sa.DateTime(), nullable=True),
        sa.Column("executed_at", sa.DateTime(), nullable=True),
    )

def downgrade() -> None:
    op.drop_table("action_proposals")
```

11.3 — Montage du router

```
# api/router.py – une ligne ajoutée, même schéma que rag/agent/connectors
from app.gating.router import router as gating_router
```

```
api_router.include_router(gating_router, prefix="/gating", tags=["gating"])
```

11.4 — Checklist de mise en œuvre

- ☐ US-401 — `gating/models.py` + migration Alembic
- ☐ US-402 — `gating/policy.py` + 2 réglages `config.py` + tests des 3 décisions
- ☐ US-403 — `gating/queue_service.py`, `gating/router.py`, squelette `audit/service.py`
- ☐ US-404 — `agent/tools/send_email.py`, attribut `sensitive` sur `Tool`
- ☐ US-405 — nœud `call_tools` étendu, reprise via `Command(resume=...)`
- ☐ US-406 — bascule `GATING_POLICY` documentée dans le `README.md`

Ce qui vient ensuite

Où ce module s'arrête volontairement, et ce que les épics suivants viennent compléter.

12.1 — Epic 5 : audit complet

Le squelette `audit/service.py` proposé au module 07 se limite à un log structuré. L'Epic 5 le remplace par une vraie table `AuditLog` persistante (US-501), journalise en plus chaque appel LLM et chaque décision de gating de façon systématique (US-502), et expose `GET /audit/logs` pour la consultation (US-503) — sans qu'aucun appelant de `write_log()` n'ait besoin de changer.

12.2 — Epic 6 : rendre le gating démontrable visuellement

US-602 demande une vue listant les `ActionProposal` en attente avec un bouton approuver/rejeter en un clic — une interface directe au-dessus de `GET /gating/pending` et `POST /gating/{id}/decide`, déjà entièrement construits par ce manuel. Aucun nouveau concept métier : uniquement de la présentation.

Aspect	Aujourd'hui (fin Epic 4)	Demain
Journalisation	Log structuré uniquement	Table <code>AuditLog</code> persistante + API de consultation (Epic 5)
Validation humaine	Via <code>curl</code> / Swagger UI sur <code>/gating/*</code>	Tableau de bord cliquable (Epic 6, US-602)
Deuxième outil sensible	Aucun	Tout nouvel outil hérite du même mécanisme en ne fixant que <code>sensitive = True</code>

🔪 NOTE — LA PREUVE DE RÉUTILISABILITÉ TIENT EN UNE LIGNE

Ajouter un futur outil sensible (créer un ticket, déclencher un remboursement) ne demandera **ni** nouveau nœud LangGraph, **ni** nouvelle route HTTP : seulement une nouvelle sous-classe de `Tool` avec `sensitive = True`, et une ligne dans `GATING_POLICY`. C'est exactement le pari architectural du module 01.

§ GLOSSAIRE COMPLET — DE A À Z

Seuls les termes introduits ou redéfinis par ce manuel figurent ici. Pour les termes déjà couverts (*checkpointer*, *thread*, *reducer*, *State Graph*...), voir le glossaire du `orchestrator-field-manual.md` (Epic 3).

ActionProposal — l'enregistrement d'une action que l'agent souhaite exécuter, portant son statut (*pending/approved/rejected/executed*) de sa création à son issue finale.

action sensible — une action dont l'exécution produit un effet de bord réel et potentiellement coûteux à annuler, par opposition à une action en lecture seule.

auto_execute — décision de politique où une action s'exécute immédiatement, sans attendre de validation humaine, après création directe d'une *ActionProposal* au statut approuvé.

Command(resume=...) — objet *LangGraph* permettant de reprendre un graphe interrompu en réinjectant une valeur exactement à l'endroit où `interrupt()` avait suspendu l'exécution.

fail safe — principe de conception où, face à une configuration absente ou ambiguë, le système retient systématiquement le comportement le moins risqué.

garde-fou (guardrail) — une vérification additionnelle restreignant un comportement par ailleurs autorisé, par exemple un seuil de confiance minimal.

human-in-the-loop — un système où une étape automatisée requiert une décision humaine explicite avant de se poursuivre.

idempotence — propriété d'une opération qui produit le même résultat qu'elle soit exécutée une ou plusieurs fois — critique ici car le nœud interrompu est rejoué depuis son début à la reprise.

interrupt() (dynamique) — fonction *LangGraph* appelée depuis l'intérieur d'un nœud, qui suspend son exécution, persiste l'état via le *checkpointer*, et renvoie la main à l'appelant avec une valeur donnée.

interrupt_before (statique) — option de compilation de graphe forçant une pause systématique avant un nœud nommé, quel que soit le contenu traité — moins granulaire que `interrupt()`.

machine à états — modélisation d'une entité ne pouvant se trouver que dans un nombre fini de statuts, avec des transitions explicites entre eux.

outil sensible — synonyme d'action sensible, appliqué à l'attribut `Tool.sensitive`.

Ports & Adapters (rappel) — ici illustré par `audit.service.write_log()` : `gating` dépend d'un contrat, jamais d'une implémentation concrète, ce qui permet à l'Epic 5 de la remplacer sans casser l'Epic 4.

rejeu (replay) — ré-exécution du code d'un nœud depuis son début lors d'une reprise, jusqu'à atteindre le point d'interruption — d'où l'exigence d'idempotence.

require_validation — décision de politique où une *ActionProposal* est créée au statut *pending* et le graphe interrompu, en attente d'une décision humaine explicite.

squelette minimal (stub) — implémentation volontairement incomplète mais fonctionnelle d'une interface, appelée à être enrichie plus tard sans rupture de contrat.

StrEnum — énumération Python dont chaque membre est simultanément une chaîne de caractères, simplifiant la sérialisation JSON et le stockage en base.

suggest_only — décision de politique la plus prudente : l'agent formule une suggestion en langage naturel sans créer d'*ActionProposal* ni produire aucun effet de bord.

MindOps — Manuel du Gating · Edition 1.0

Documentation Epic 4 · Usage interne · Architecture anticipée, à valider lors de l'implémentation réelle